

Temporal Action Detection on THUMOS-14 Using Pre-Extracted I3D Features

Vlad Lagutin

1 Introduction

Temporal action detection (TAD) is different from ordinary classification in a way that its goal is not to just identify *what* actions happen in the video, but also *when* they happen, outputting a class label along with a specific start and end time for each action. This task is much more complicated than default classification - a model must decide, for every moment in a video, whether an action is occurring and which one it is, considering most of the frames in a video contain no action altogether (background).

The THUMOS-14 benchmark (Jiang et al. 2014) is a popular dataset for this problem. It contains 20 sport action classes - *BasketballDunk*, *CliffDiving*, *GolfSwing*, etc. - annotated with frame-level timestamps across 412 untrimmed videos. Actions are short (about 4.3 secs, on average), sometimes overlapping, and occur against a heavy background, which makes the problem truly hard.

Training video models end-to-end requires gigantic computing power and memory. A popular alternative is the following approach: first extract compact per-frame feature vectors using a pre-trained model, then train a classifier on them. I am going to stick to this strategy and use publicly available I3D features (Carreira and Zisserman 2017), which capture both appearance and short-range motion in a 2048-dimensional vector per feature step. This significantly simplifies the problem and lets me focus on the sequence modeling to answer the following question: which classifier architecture best leverages temporal structure in the features?

I train and compare four different architectures that I've learned in class - MLP, 1D-CNN, LSTM, and Transformer. Under identical conditions, I plan to evaluate both frame-level classification accuracy and the quality of temporal detection.

2 Background

I3D features. The Inflated 3D ConvNet (I3D) (Carreira and Zisserman 2017) extends standard 2D image CNNs to video by inflating convolution kernels into 3D, allowing the model to learn both spatial appearance and temporal motion together. Pre-trained on the large-scale Kinetics dataset, I3D produces 2048-dimensional feature vectors at the second to last layer, one per short video chunk. These features would constitute my dataset and will be used as a representation of videos.

Evaluation metrics. Frame-level classification is measured with accuracy and macro-averaged F1, which treats all classes equally regardless of frequency. For temporal detection I use mean Average Precision at $\text{IoU} \geq 0.5$ (mAP@0.5), which is the standard metric for THUMOS-14. *Intersection over Union* (IoU) measures how well a predicted segment temporally overlaps with a true segment: it is the duration of their overlap divided by the duration of their union. A prediction counts as correct only if its IoU with some true instance exceeds the threshold (here 0.5). *Average Precision* (AP) is the area under the precision-recall curve for one class; mAP averages AP across all classes.

3 Methods

3.1 Dataset and Split

THUMOS-14 provides temporal annotations for 412 videos: a *validation* split (200 videos) and a *test* split (212 videos). Common practice is to treat the validation videos as our labeled training data and leave the test split for final evaluation only. The 200 training videos were randomly split 80/20 into a training set (160 videos) and a validation set (40 videos), which was used for early stopping and model selection.

The I3D features were extracted at a stride of 4 frames (from 30 fps video), giving ~ 7.5 feature vectors per second. Each feature vector has dimension 2048. The final labeled dataset contains over 330,000 timesteps.

3.2 Class Imbalance

This dataset is extremely imbalanced. Background accounts for roughly 182,000 of the training timesteps, while rare classes like *BaseballPitch* appear in only ~ 800 . To prevent models predicting exclusively background, I used **Inverse-frequency class weights** on the cross-entropy loss (for CNN and LSTM), so each class contributes to the loss in proportion to how rare it is.

3.3 Model Architectures

All models map 2048-dimensional I3D vectors to one of 21 class labels (20 actions + background). Each is trained for up to 30 epochs with early stopping (patience = 5 epochs on validation macro-F1), AdamW optimizer ($lr = 10^{-3}$, weight decay 10^{-4}), and a ReduceLROnPlateau learning rate scheduler (factor 0.3, patience 3).

MLP. Treats each timestep as a completely independent observation, with no knowledge of neighboring frames. Four linear layers ($2048 \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 21$) with ReLU activations and dropout (0.25). This is a context-free baseline.

1D-CNN. Model based on sliding window where the middle frame of a 15-frame window (± 7 frames, ~ 1 second of context) is being classified. Two 1D convolutional layers (kernel size 3, 256 channels) with batch normalization and ReLU, followed by a linear head.

LSTM. A two-layer one-directional LSTM (hidden size 256, dropout 0.3) that reads the same 15-frame window, then uses the final hidden state to classify the center frame. The recurrence is designed to capture the order of temporal dependencies that are not captured by CNN.

Transformer. A Transformer encoder with an input projection ($2048 \rightarrow 256$), 2 encoder layers (4 attention heads, feedforward dim 1024, dropout 0.1). As opposed to models that operate on individual windows, this model attends to the whole video sequence using self-attention and performs classification on the central frame.

3.4 Temporal Proposal Generation

For detection, I construct temporal proposals based on the per-frame softmax probabilities through an elementary thresholding and merging technique: for each non-background category, I compute the timesteps that have a predicted probability higher than θ , merge consecutive positives into segments, and record the mean probability as the score of this segment. I use $\theta \in \{0.2, 0.3, 0.4, 0.5\}$ and evaluate each with the standard THUMOS mAP@0.5 metric.

4 Experiments

4.1 Frame-Level Classification

Table 1 summarizes the three key metrics for all four models.

Table 1: Frame-level classification results on the THUMOS-14 test set.

Model	Best Val F1	Test Accuracy	Test Macro F1
MLP	0.416	0.641	0.477
CNN	0.484	0.756	0.547
LSTM	0.506	0.723	0.542
Transformer	0.402	0.559	0.459

The CNN and LSTM significantly outperform the MLP, showing that even a temporal receptive field of ~ 1 second can notably improve classification. The CNN achieves the highest test accuracy (75.6%), while the LSTM leads on validation macro-F1 (0.506) - the metric used for model selection. The Transformer underperforms despite its global receptive field. Probable reasons: (1) it was trained on a windowed data loader that does not fully exploit its whole-sequence attention; and (2) it has a higher number of parameters, making it difficult to train with few video samples.

Figure 1 shows training curves for all four models. All of them overfit noticeably - training metrics diverge from validation early. Even LSTM and Transformer fail to avoid overfitting, despite their intrinsic regularizers.

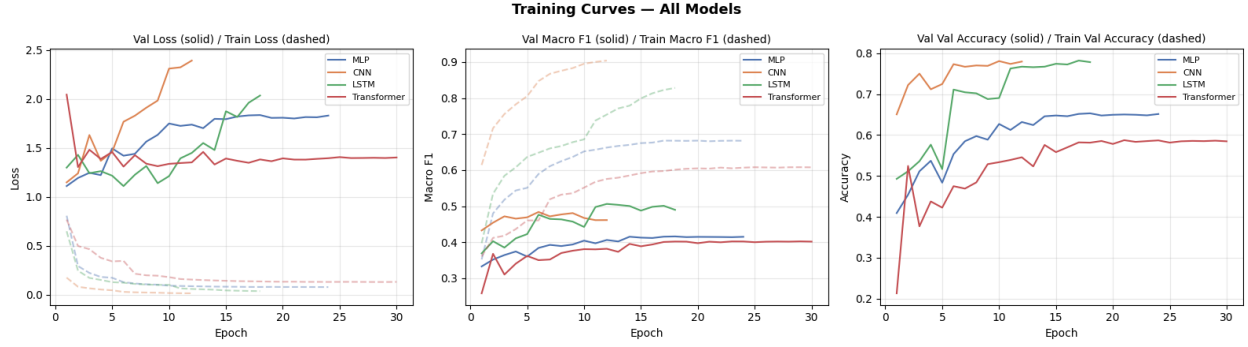


Figure 1: Training curves: validation (solid) and training (dashed) for loss, macro-F1, and accuracy across 30 epochs.

4.1.1 Per-Class Performance

Figure 2 shows per-class test F1 across all models. Actions with long duration and high visibility - *BasketballDunk*, *CliffDiving*, *CleanAndJerk*, *PoleVault* - consistently score highest ($F1 \geq 0.65$). Short and visually ambiguous classes like *BaseballPitch*, *CricketShot*, *JavelinThrow* fall below 0.40 across all models. The fact that this trend is consistent across all models implies that the limitation lies in the feature extraction level (I3D features are not discriminative enough for these classes) and not architecture.

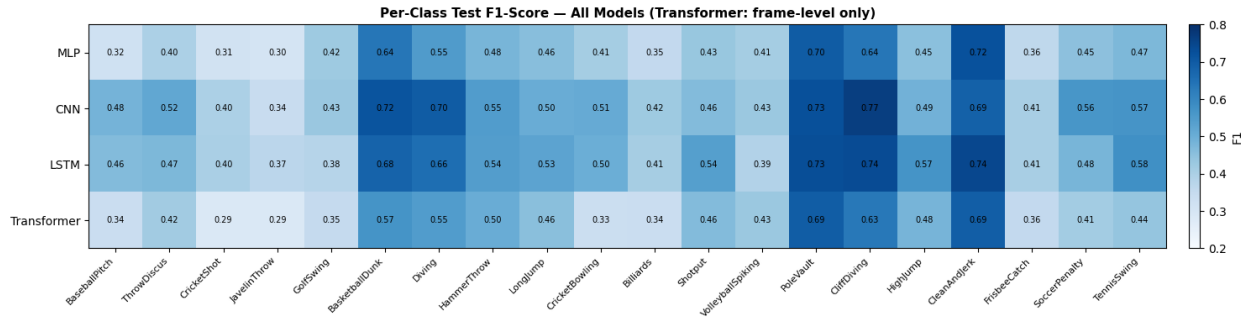


Figure 2: Per-class test F1 for all four models. No transformer because I failed to calculate these metrics for it (couldn't fix it). :(

4.2 Temporal Detection

Table 2 shows detection $mAP@0.5$ at the best confidence threshold for each model.

Table 2: Temporal detection results ($mAP@0.5$ at best confidence threshold).

Model	$mAP@0.5$	Best θ
MLP	0.055	0.5
CNN	0.089	0.5
LSTM	0.088	0.5

The CNN and LSTM achieve nearly identical detection performance (~ 0.089 and ~ 0.088 mAP), both considerably better than MLP (0.055). The gap between frame-level F1 (~ 0.55) and detection mAP (~ 0.09) is huge. This is expected: for mAP@0.5, it is essential to detect not only the correct class but also for the boundary segments predicted to overlap temporally with the true label for at least 50%. My simple threshold-and-merge approach makes no effort to refine boundaries, so even well-classified actions can output proposals that miss the IoU threshold.

Figure 3 illustrates how rapidly mAP degrades as the IoU threshold increases from 0.3 to 0.5, highlighting the challenge of localization.

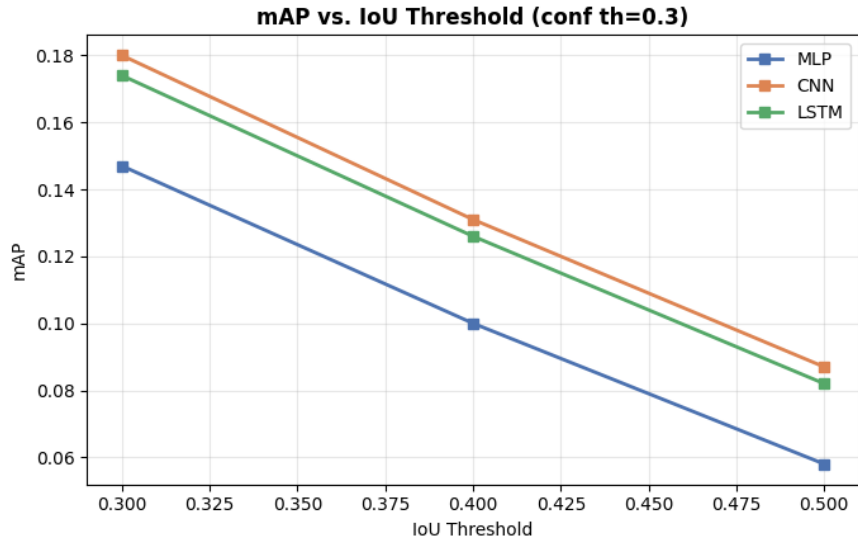


Figure 3: mAP vs. IoU threshold (confidence $\theta = 0.3$).

4.2.1 Per-Class Detection

Figure 4 breaks detection down by class at IoU = 0.5. The highest scoring actions include *CliffDiving* (LSTM: 0.18), *GolfSwing* (LSTM: 0.16), and *Shotput/ThrowDiscus* (CNN: 0.15), which are long actions with distinct motion patterns. *FrisbeeCatch*, *SoccerPenalty*, and *TennisSwing* score near zero for all models, reflecting their short duration (making precise localization extremely important) and visual overlap with other classes.

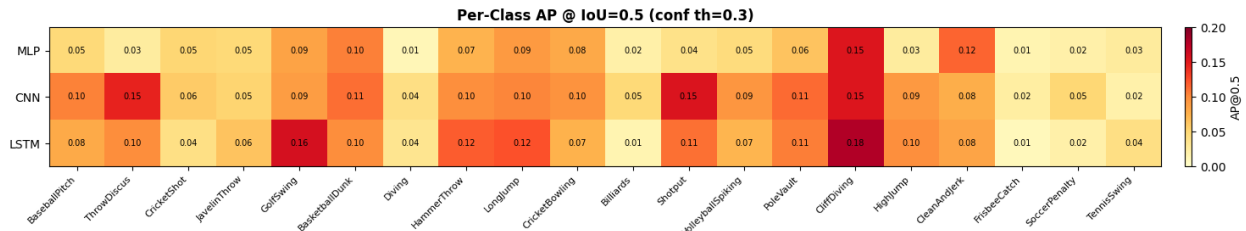


Figure 4: Per-class AP@IoU=0.5 ($\theta = 0.3$) for MLP, CNN, and LSTM.

5 Discussion

The results tell a consistent story. **Temporal context is the most important aspect:** the CNN and LSTM, which each see about one second of context performed significantly better than context-free MLP in both classification and detection. Within the windowed models, differences are quite small.

The results by the Transformer demonstrate once again that simply having a large capacity network does not guarantee good results if there is less amount of data (around 160 videos). It needs more data to achieve its full potential.

Detection mAP is low across all models (~ 0.05 – 0.09), highlighting that accurate frame classification is only part of the problem. Boundary localization is the another part, which requires far more than just a threshold-and-merge method. Proposal generation methods like BSN (Lin et al. 2018) specifically model start and end boundaries and would be one of the most expected extensions of this project.

References

- Carreira, João, and Andrew Zisserman. 2017. “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset.” In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. <https://arxiv.org/pdf/1705.07750>.
- Jiang, Y.-G., J. Liu, A. Roshan Zamir, G. Toderici, I. Laptev, M. Shah, and R. Sukthankar. 2014. “THUMOS Challenge 2014.” <http://crcv.ucf.edu/THUMOS14/>.
- Lin, Tianwei, Xu Zhao, Haisheng Su, Chongjing Wang, and Ming Yang. 2018. “BSN: Boundary Sensitive Network for Temporal Action Proposal Generation.” In *European Conference on Computer Vision (ECCV)*. <https://arxiv.org/pdf/1806.02964>.